

Sprint 8 — Facade Pattern

Class 1

Learning Objectives

By the end of this class, we should be able to:

- Explain why Facade Pattern exists.
- Recognize when a client knows too much about a subsystem.
- Create a simple Facade.
- Coordinate multiple services through one entry point.
- Understand how Facade improves readability and maintainability.
- Avoid turning a Facade into a God Class.

Lesson 1 — Why Facade Exists

Start With a Real Enterprise Problem

Imagine we are developing an online shopping system.

When a customer places an order, the system must:

1. Validate the customer
2. Validate the order
3. Check inventory
4. Reserve products
5. Process payment
6. Create shipment
7. Generate invoice
8. Send confirmation
9. Write audit log

This is not one simple operation.

It involves many subsystems.

Bad Design

```
public class OrderController {

    private final CustomerService customerService;
    private final OrderValidator orderValidator;
    private final InventoryService inventoryService;
    private final PaymentService paymentService;
    private final ShippingService shippingService;
    private final InvoiceService invoiceService;
    private final NotificationService notificationService;
    private final AuditService auditService;

    public OrderController(
        CustomerService customerService,
        OrderValidator orderValidator,
        InventoryService inventoryService,
        PaymentService paymentService,
        ShippingService shippingService,
        InvoiceService invoiceService,
        NotificationService notificationService,
        AuditService auditService
    ) {
        this.customerService = customerService;
        this.orderValidator = orderValidator;
        this.inventoryService = inventoryService;
        this.paymentService = paymentService;
        this.shippingService = shippingService;
        this.invoiceService = invoiceService;
        this.notificationService = notificationService;
        this.auditService = auditService;
    }

    public void placeOrder(OrderRequest request) {
        customerService.validateCustomer(request.customerId());
    }
}
```

```
orderValidator.validate(request);

inventoryService.checkAvailability(request.items());

inventoryService.reserve(request.items());

PaymentResult paymentResult =
    paymentService.processPayment(request.payment());

Shipment shipment =
    shippingService.createShipment(request);

Invoice invoice =
    invoiceService.generateInvoice(request);

notificationService.sendOrderConfirmation(
    request.customerId(),
    invoice
);

auditService.recordOrderCreated(
    request,
    paymentResult,
    shipment
);
}
}
```

Question

What is wrong with this controller?

Possible answers:

- It knows too many services.
- It knows the complete workflow.

- It knows the order in which services must be called.
- It is difficult to test.
- It is difficult to reuse.
- Business workflow is placed in the controller.
- A change to the workflow requires changing the controller.

The controller knows too much.

What Happens When Requirements Change?

Business says:

Add fraud checking before payment.

We modify OrderController.

Later:

Add loyalty points after successful payment.

Modify OrderController.

Later:

Send Telegram notification.

Modify OrderController again.

Eventually, the controller becomes responsible for coordinating the entire business process.

That is not the controller's responsibility.

Lesson 2 — What Is Facade Pattern?

Simple Definition

Facade Pattern provides a simple interface to a complex subsystem.

Easy sentence for us:

Facade hides complexity behind one simple method.

Instead of this:

Controller

- CustomerService
- OrderValidator
- InventoryService
- PaymentService
- ShippingService
- InvoiceService
- NotificationService
- AuditService

We create:

Controller

|

OrderFacade

- CustomerService
- OrderValidator
- InventoryService
- PaymentService
- ShippingService
- InvoiceService
- NotificationService
- AuditService

The controller now knows only one object:

OrderFacade

Facade Structure

Client

|

Facade

|

—— Subsystem A

—— Subsystem B

—— Subsystem C

—— Subsystem D

Main Participants

Client

The class that needs to use the subsystem.

Example: OrderController

Facade

Provides the simple entry point.

Example: OrderFacade

Subsystems

Classes that perform the real work.

Examples:

InventoryService

PaymentService

ShippingService

InvoiceService

NotificationService

The Facade does not replace these services.

It coordinates them.

Lesson 3 — Create the Subsystem

For classroom demonstration, keep every subsystem simple.

Customer Service

```
public class CustomerService {  
  
    public void validateCustomer(String customerId) {  
        System.out.println(  
            "Validate customer: " + customerId  
        );  
    }  
}
```

Order Validator

```
public class OrderValidator {  
  
    public void validate(OrderRequest request) {  
        System.out.println(  
            "Validate order: " + request.orderNumber()  
        );  
    }  
}
```

Inventory Service

```
public class InventoryService {  
  
    public void checkAvailability(OrderRequest request) {  
        System.out.println(  
            "Check product availability"  
        );  
    }  
}
```

```

    );
}

public void reserve(OrderRequest request) {
    System.out.println(
        "Reserve products"
    );
}
}

```

Payment Service

```

public class PaymentService {

    public PaymentResult processPayment(
        OrderRequest request
    ) {
        System.out.println(
            "Process payment: " + request.totalAmount()
        );

        return new PaymentResult(
            "PAY-1001",
            true
        );
    }
}

```

Shipping Service

```

public class ShippingService {

    public Shipment createShipment(
        OrderRequest request
    ) {
        System.out.println(
            "Create shipment"
        );
    }
}

```

```
        return new Shipment(  
            "SHIP-1001"  
        );  
    }  
}
```

Invoice Service

```
public class InvoiceService {  
  
    public Invoice generateInvoice(  
        OrderRequest request  
    ) {  
        System.out.println(  
            "Generate invoice"  
        );  
  
        return new Invoice(  
            "INV-1001"  
        );  
    }  
}
```

Notification Service

```
public class NotificationService {  
  
    public void sendConfirmation(  
        String customerId,  
        Invoice invoice  
    ) {  
        System.out.println(  
            "Send confirmation to customer "  
            + customerId  
        );  
    }  
}
```

Audit Service

```
public class AuditService {  
  
    public void recordOrderCreated(  
        OrderRequest request  
    ) {  
        System.out.println(  
            "Write audit log for "  
                + request.orderNumber()  
        );  
    }  
}
```

Lesson 4 — Create the Facade

```
public class OrderFacade {  
  
    private final CustomerService customerService;  
    private final OrderValidator orderValidator;  
    private final InventoryService inventoryService;  
    private final PaymentService paymentService;  
    private final ShippingService shippingService;  
    private final InvoiceService invoiceService;  
    private final NotificationService notificationService;  
    private final AuditService auditService;  
  
    public OrderFacade(  
        CustomerService customerService,  
        OrderValidator orderValidator,  
        InventoryService inventoryService,  
        PaymentService paymentService,  
        ShippingService shippingService,  
        InvoiceService invoiceService,  
        NotificationService notificationService,  
        AuditService auditService
```

```
) {  
    this.customerService = customerService;  
    this.orderValidator = orderValidator;  
    this.inventoryService = inventoryService;  
    this.paymentService = paymentService;  
    this.shippingService = shippingService;  
    this.invoiceService = invoiceService;  
    this.notificationService = notificationService;  
    this.auditService = auditService;  
}  
  
public PlaceOrderResult placeOrder(  
    OrderRequest request  
) {  
    customerService.validateCustomer(  
        request.customerId()  
    );  
  
    orderValidator.validate(request);  
  
    inventoryService.checkAvailability(request);  
  
    inventoryService.reserve(request);  
  
    PaymentResult payment =  
        paymentService.processPayment(request);  
  
    Shipment shipment =  
        shippingService.createShipment(request);  
  
    Invoice invoice =  
        invoiceService.generateInvoice(request);  
  
    notificationService.sendConfirmation(  
        request.customerId(),  
        invoice  
    );  
}
```

```
        auditService.recordOrderCreated(request);

        return new PlaceOrderResult(
            request.orderNumber(),
            payment.transactionId(),
            shipment.shipmentNumber(),
            invoice.invoiceNumber(),
            "ORDER_CREATED"
        );
    }
}
```

Important Point

The Facade does not contain the detailed implementation of:

- Payment
- Inventory
- Shipping
- Invoice
- Notification

It only coordinates the workflow.

The real work remains inside focused subsystem classes.

Lesson 5 — Client After Facade

Controller Before

```
customerService.validateCustomer(...);
orderValidator.validate(...);
inventoryService.checkAvailability(...);
inventoryService.reserve(...);
paymentService.processPayment(...);
```

```
shippingService.createShipment(...);
invoiceService.generateInvoice(...);
notificationService.sendConfirmation(...);
auditService.recordOrderCreated(...);
```

Controller After

```
public class OrderController {

    private final OrderFacade orderFacade;

    public OrderController(
        OrderFacade orderFacade
    ) {
        this.orderFacade = orderFacade;
    }

    public PlaceOrderResult placeOrder(
        OrderRequest request
    ) {
        return orderFacade.placeOrder(request);
    }
}
```

Much simpler.

Demo

```
import java.math.BigDecimal;

public class FacadePatternDemo {

    public static void main(String[] args) {

        OrderFacade facade = new OrderFacade(
            new CustomerService(),
            new OrderValidator(),
```

```
        new InventoryService(),
        new PaymentService(),
        new ShippingService(),
        new InvoiceService(),
        new NotificationService(),
        new AuditService()
    );

    OrderRequest request = new OrderRequest(
        "ORDER-2026-0001",
        "CUSTOMER-001",
        new BigDecimal("150.00")
    );

    PlaceOrderResult result =
        facade.placeOrder(request);

    System.out.println();
    System.out.println(result);
}
}
```

Output

```
Validate customer: CUSTOMER-001
Validate order: ORDER-2026-0001
Check product availability
Reserve products
Process payment: 150.00
Create shipment
Generate invoice
Send confirmation to customer CUSTOMER-001
Write audit log for ORDER-2026-0001

PlaceOrderResult[
    orderNumber=ORDER-2026-0001,
    paymentTransactionId=PAY-1001,
```

```
shipmentNumber=SHIP-1001,  
invoiceNumber=INV-1001,  
status=ORDER_CREATED  
]
```

Lesson 6 — What Problem Did We Solve?

Before Facade

The client knows:

- Which services exist
- Which method to call
- Which order to call them
- How to pass results between services
- What to do when the workflow changes

After Facade

The client knows only:

```
orderFacade.placeOrder(request);
```

The Facade knows the workflow.

Subsystem services know their own responsibilities.

Lesson 7 — SOLID Connection

Single Responsibility Principle

The controller handles HTTP concerns.

The Facade coordinates the use case.

Subsystem services perform specialized work.

Example:

OrderController

Responsibility: HTTP request and response

OrderFacade

Responsibility: coordinate order placement

PaymentService

Responsibility: payment processing

InventoryService

Responsibility: inventory operations

Dependency Inversion Principle

The controller depends on a higher-level entry point:

OrderFacade

It does not depend on every low-level subsystem.

In a stronger enterprise design, the controller may depend on an interface:

```
public interface PlaceOrderUseCase {
```

```
    PlaceOrderResult placeOrder(
        OrderRequest request
    );
}
```

Implementation:

```
public class OrderFacade
    implements PlaceOrderUseCase {
}
```

Then the controller depends on:

PlaceOrderUseCase

Open/Closed Principle

When adding a new subsystem step, the controller does not change.

Example:

FraudCheckService

LoyaltyPointService

TelegramNotificationService

The Facade coordinates the updated workflow.

However, note that changing the workflow may still require modifying the Facade.

That is not automatically an OCP violation.

The Facade's responsibility is the workflow, so workflow changes are valid reasons for it to change.

Lesson 8 — Facade vs God Class

This lesson is very important.

We often misunderstand Facade and create one giant class.

Bad Facade

```
public class ApplicationFacade {  
  
    public void registerUser() {  
    }  
  
    public void login() {  
    }  
  
    public void resetPassword() {  
    }  
  
    public void createOrder() {  
    }  
}
```

```
public void pay() {  
}  
  
public void createShipment() {  
}  
  
public void generateReport() {  
}  
  
public void uploadFile() {  
}  
  
public void approveRoom() {  
}  
}
```

This is not a good Facade.

It is becoming a God Class.

Good Facades

RegistrationFacade

AuthenticationFacade

CheckoutFacade

RoomPublishingFacade

SubscriptionFacade

Each Facade represents one focused workflow or subsystem.

Important Rule

A Facade should simplify a focused use case.

It should not become the entire application.

Lesson 9 — Real Enterprise Example

Room Publishing Facade

Suppose an owner wants to publish a room.

The workflow may require:

1. Load room
2. Verify ownership
3. Verify property status
4. Check active subscription
5. Validate room data
6. Validate uploaded media
7. Change room status
8. Create moderation case
9. Publish event

Without Facade, the controller may know all these services.

With Facade:

```
public class RoomPublishingFacade {  
  
    private final RoomReader roomReader;  
    private final OwnershipValidator ownershipValidator;  
    private final PropertyClient propertyClient;  
    private final SubscriptionClient subscriptionClient;  
    private final RoomValidator roomValidator;  
    private final MediaClient mediaClient;  
    private final RoomWriter roomWriter;  
    private final ModerationClient moderationClient;  
    private final RoomEventPublisher eventPublisher;  
  
    public RoomPublishingResult publish(  
        String roomId,  
        String ownerId  
    ) {  
        Room room = roomReader.getRequired(roomId);  
  
        ownershipValidator.validate(  
            room,
```

```

        ownerId
    );

    propertyClient.verifyPublishable(
        room.propertyId()
    );

    subscriptionClient.verifyActive(
        ownerId
    );

    roomValidator.validateForPublishing(
        room
    );

    mediaClient.verifyFiles(
        room.mediaIds()
    );

    Room publishedRoom =
        roomWriter.markSubmitted(room);

    moderationClient.createCase(
        publishedRoom.id()
    );

    eventPublisher.publishSubmitted(
        publishedRoom
    );

    return new RoomPublishingResult(
        publishedRoom.id(),
        publishedRoom.status()
    );
}
}

```

Controller:

```
@PostMapping("/{roomId}/submit")
public RoomPublishingResult submit(
    @PathVariable String roomId,
    Authentication authentication
) {
    return roomPublishingFacade.publish(
        roomId,
        authentication.getName()
    );
}
```

The controller remains clean.

Common Mistakes

Mistake 1 — Business Logic Everywhere in the Facade

Bad:

```
if (customer.isVip()
    && amount.compareTo(...) > 0) {
    // large discount calculation
}
```

Complex domain rules should normally belong in domain services, policies, or strategies.

The Facade coordinates them.

Mistake 2 — Exposing Every Subsystem Method

Bad Facade:

```
public void validateCustomer() {
}

public void reserveInventory() {
}
```

```
public void processPayment() {  
}
```

The client still has to understand the workflow.

Better:

```
public PlaceOrderResult placeOrder(  
    OrderRequest request  
) {  
}
```

One meaningful operation.

Mistake 3 — One Facade for the Entire Application

Avoid:

ApplicationFacade

Prefer focused Facades.

Mistake 4 — Replacing Every Service with Facade

Facade does not eliminate subsystem services.

It provides a simpler entry point over them.

Mistake 5 — Returning Third-Party or Internal Subsystem Models

The Facade should return application-level results.

Good:

PlaceOrderResult

Avoid returning:

StripeChargeResponse

InventoryReservationEntity

ShippingSdkResponse

Class 1 Summary

Concept	Meaning
Facade	Simple interface to a complex subsystem
Client	Uses the Facade
Subsystem	Performs specialized work
Main benefit	Client does not need to understand subsystem complexity
Facade responsibility	Coordinate a focused workflow
Important warning	Do not create a God Class

Homework

Hotel Booking Facade

A hotel booking requires:

1. Validate guest
2. Check room availability
3. Calculate price
4. Reserve room
5. Process payment
6. Create booking
7. Send confirmation
8. Write audit log

Create these subsystem classes:

GuestService

RoomAvailabilityService

PricingService

RoomReservationService

PaymentService

BookingService

NotificationService

AuditService

Create:

BookingFacade

The client should call only:

```
bookingFacade.book(request);
```

Homework Requirements

1. Create BookingRequest.
2. Create BookingResult.
3. Create all subsystem services.
4. Create BookingFacade.
5. Make the controller depend only on the Facade.
6. Add loyalty points without modifying the controller.
7. Explain why the Facade is not a God Class.
8. Identify where business validation should live.

Interview Questions

1. What problem does Facade Pattern solve?
2. What is the difference between Facade and Adapter?
3. Does Facade replace subsystem classes?
4. Can one application have multiple Facades?
5. How does Facade support SRP?
6. What is the difference between Facade and a God Class?
7. Should a Facade contain business rules?
8. Can a Facade call another Facade?
9. What are some enterprise examples of Facade?

10. Is a Spring @Service automatically a Facade?

End of Class 1